Binary Search Trees Lab

Data Structures & Algorithms

Lab 06 -- Building a BST from Scratch and Putting It to the Test

Duration: 2 hours (2 challenges)

Lab Structure

Each challenge follows this format:

1. **Challenge Presentation** (5 min)
2. **Your Implementation Time** (25 min)
3. **Pseudocode Review** (5 min)
4. **Solution Walkthrough** (10 min)
5. **Code Explanation** (15 min)

Total per challenge: ~60 minutes

Today's Challenges

Challenge 1: Build a Complete BST with All Operations

- Implement **insert**, **search**, **find-min**, **find-max**, and **in-order traversal** for a BST
- Feed it a scrambled list of student roll numbers
- Watch the in-order traversal produce a perfectly sorted roster -- automatically!

Challenge 2: The Scoreboard -- BST vs. Sorted Array Showdown

- Build **both** a BST and a sorted array from the same data
- Perform the same set of operations on both: search, insert, delete
- **Count the operations** each approach takes
- See with your own eyes why BSTs win when data is dynamic

Challenge 1

Build a Complete BST with All Operations

Challenge 1: Problem Statement

Scenario: You are building a student roll number lookup system. Given a scrambled list of roll numbers, build a BST that can:

1. Insert new roll numbers
2. Search for a roll number (is this student enrolled?)
3. Find the smallest and largest roll numbers
4. Print all roll numbers in sorted order (in-order traversal)

Test data (insert in this order): 50, 30, 70, 20, 40, 60, 80, 10, 25, 35

Requirements:

- Define a BST node struct with `data`, `left`, `right`
- Implement `insert(root, value)` -- returns the updated root
- Implement `search(root, key)` -- returns pointer to node or NULL
- Implement `findMin(root)` and `findMax(root)`
- Implement `inorder(root)` -- prints sorted output
- Test: insert all values, search for 40 (exists) and 55 (missing), find min/max

Time: 25 minutes

Challenge 1: Think About It First

Before coding, trace the tree that forms when we insert 50, 30, 70, 20, 40, 60, 80, 10, 25, 35:

```
Insert 50:  50          Insert 30:   50         Insert 70:   50
                                    /                       /  \
                                  30                      30    70

Insert 20:   50         Insert 40:    50        ... (continue building)
            /  \                      /  \
          30    70                  30    70
         /                          / \
       20                         20  40
```

Final tree:

```
                  50
                /    \
              30      70
             / \      / \
           20   40  60  80
          /  \  /
        10  25 35
```

In-order traversal should give: 10, 20, 25, 30, 35, 40, 50, 60, 70, 80 -- sorted!

Challenge 1: Pseudocode

```
ALGORITHM BinarySearchTree

  1. Define Node: int data, pointer left, pointer right

  2. createNode(value):
     - Allocate, set data = value, left = right = NULL, return it

  3. insert(root, value):
     - If root is NULL: return createNode(value)
     - If value < root->data: root->left = insert(root->left, value)
     - If value > root->data: root->right = insert(root->right, value)
     - Return root

  4. search(root, key):
     - If root is NULL: return NULL (not found)
     - If key == root->data: return root
     - If key < root->data: return search(root->left, key)
     - Else: return search(root->right, key)

  5. findMin(root): go left until left is NULL, return that node
  6. findMax(root): go right until right is NULL, return that node
  7. inorder(root): if not NULL -> inorder(left), print data, inorder(right)

END ALGORITHM
```

Challenge 1: Solution Code -- Node and Insert

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) { printf("Memory error!\n"); exit(1); }
    newNode->data  = value;
    newNode->left  = NULL;
    newNode->right = NULL;
    return newNode;
}

struct Node* insert(struct Node* root, int value) {
    if (root == NULL) return createNode(value);
    if (value < root->data)
        root->left  = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    // If value == root->data, it is a duplicate -- ignore
    return root;
}
```

Challenge 1: Solution Code -- Search and Min/Max

```c
// Search for a value in the BST
struct Node* search(struct Node* root, int key) {
    if (root == NULL) return NULL;          // Not found
    if (key == root->data) return root;     // Found!
    if (key < root->data)
        return search(root->left, key);     // Go left
    else
        return search(root->right, key);    // Go right
}

// Find the node with the smallest value
struct Node* findMin(struct Node* root) {
    if (root == NULL) return NULL;
    while (root->left != NULL)
        root = root->left;                  // Keep going left
    return root;
}

// Find the node with the largest value
struct Node* findMax(struct Node* root) {
    if (root == NULL) return NULL;
    while (root->right != NULL)
        root = root->right;                 // Keep going right
    return root;
}
```

Challenge 1: Solution Code -- Traversal and Main

```c
void inorder(struct Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

int main() {
    printf("=== Challenge 1: Student Roll Number BST ===\n\n");
    struct Node* root = NULL;
    int rolls[] = {50, 30, 70, 20, 40, 60, 80, 10, 25, 35};
    int n = 10;

    for (int i = 0; i < n; i++) {
        root = insert(root, rolls[i]);
        printf("Inserted %d\n", rolls[i]);
    }

    printf("\nSorted roll numbers (in-order): ");
    inorder(root);
    printf("\n");

    int key1 = 40, key2 = 55;
    printf("\nSearch for %d: %s\n", key1, search(root, key1) ? "FOUND" : "NOT FOUND");
    printf("Search for %d: %s\n", key2, search(root, key2) ? "FOUND" : "NOT FOUND");
    printf("\nSmallest roll: %d\n", findMin(root)->data);
    printf("Largest roll:  %d\n", findMax(root)->data);
    return 0;
}
```

Challenge 1: Expected Output

```
=== Challenge 1: Student Roll Number BST ===

Inserted 50
Inserted 30
Inserted 70
...
Inserted 35

Sorted roll numbers (in-order): 10 20 25 30 35 40 50 60 70 80

Search for 40: FOUND
Search for 55: NOT FOUND

Smallest roll: 10
Largest roll:  80
```

What just happened? We inserted 10 roll numbers in a completely random order, and the in-order traversal automatically printed them sorted. No sorting algorithm was used -- the BST's structure inherently maintains order.

Challenge 1: Code Explanation

Part 1: How Insert Finds the Right Spot

```
struct Node* insert(struct Node* root, int value) {
    if (root == NULL) return createNode(value);  // <-- landing spot!
    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    return root;
}
```

Insert for value **35** on our tree:

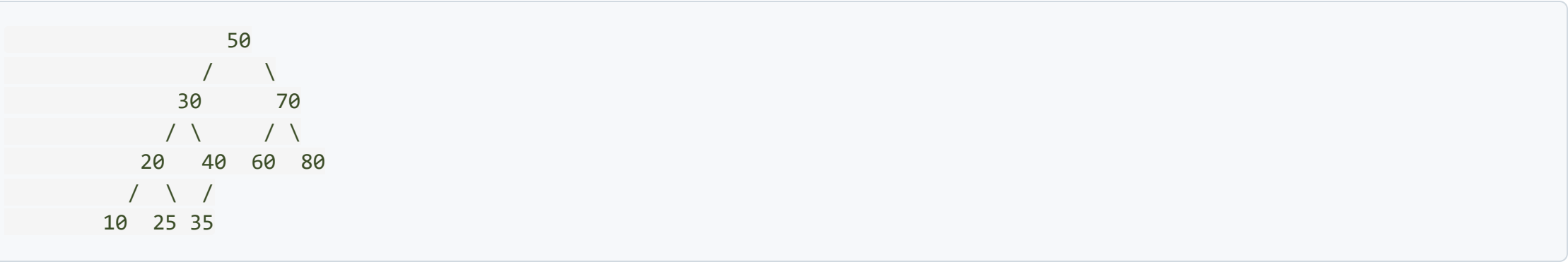
```
Start at 50: 35 < 50 --> go left
At 30:       35 > 30 --> go right
At 40:       35 < 40 --> go left
At NULL:     CREATE node 35 here!
```

The recursion returns the new node to its parent (40), which sets `root->left = newNode`. The chain of `return root` then unwinds back to the original root, keeping the tree intact.

Every new node is always a leaf. The search path determines where it lands.

Challenge 1: Code Explanation

Part 2: Why findMin Goes Left and findMax Goes Right



findMin: By the BST property, the left child is always smaller than the parent. So the smallest value is the leftmost node: 50 -> 30 -> 20 -> **10**.

findMax: Similarly, the right child is always larger. The largest is the rightmost node: 50 -> 70 -> **80**.

Both operations follow a single path down the tree -- **O(h)** where h is the height.

Compare with an unsorted array: Finding the min or max requires scanning all n elements -- O(n). A BST does it by just walking down one side.

Challenge 1: Code Explanation

Part 3: The Return Pattern -- Why `root->left = insert(root->left, value)`?

This pattern might look strange at first. Why reassign `root->left` when it might not change?

```
root->left = insert(root->left, value);
```

When root->left is NOT NULL: The recursive call walks deeper, eventually returns the unchanged subtree root. The assignment `root->left = same_value` is harmless -- it assigns the same pointer.

When root->left IS NULL: The recursive call hits the base case and returns a new node. The assignment `root->left = newNode` **attaches** the new node to the tree!

This elegant pattern appears in many tree algorithms (the same trick is used in deletion). The alternative would require tracking the parent pointer separately -- much more complex code.

Challenge 2

The Scoreboard -- BST vs. Sorted Array

Challenge 2: The Scenario

You are managing a **live leaderboard** for an online game. Players score points, and the leaderboard must stay sorted at all times. New scores arrive constantly, and players sometimes leave.

You will implement the same operations using TWO approaches:

1. A **sorted array** (using insertion sort to maintain order)
2. A **BST**

For each approach, count the number of comparisons and shifts/pointer-updates during:

- Inserting 8 scores: 50, 30, 70, 20, 40, 60, 80, 45
- Searching for score 40 (exists) and 55 (missing)
- Deleting score 30

Then compare: Which approach needed fewer operations?

Time: 25 minutes

Challenge 2: Pseudocode -- Sorted Array

```
ALGORITHM SortedArrayLeaderboard

  insert(arr, n, value):
    - Find the correct position using linear scan (count comparisons)
    - Shift all elements from that position one step to the right (count shifts)
    - Place value at the correct position
    - Increment n

  search(arr, n, key):
    - Binary search: compare with middle, go left or right half
    - Count comparisons, return index or -1

  delete(arr, n, key):
    - Find key using linear search (count comparisons)
    - Shift all elements after the key one step to the left (count shifts)
    - Decrement n

END ALGORITHM
```

Note: We use linear scan for insert position and binary search for lookup -- this mirrors how real sorted arrays work (insertion is O(n) due to shifting, search is O(log n)).

Challenge 2: Pseudocode -- BST

```
ALGORITHM BSTLeaderboard

  insert(root, value):
    - Walk down comparing at each node (count comparisons)
    - Create leaf node at the NULL position
    - No shifting needed!

  search(root, key):
    - Walk down comparing at each node (count comparisons)
    - Return node or NULL

  delete(root, key):
    - Find the node (count comparisons)
    - Case 1 (leaf): just remove
    - Case 2 (one child): promote child
    - Case 3 (two children): find in-order successor,
      copy its value, delete the successor

END ALGORITHM
```

Challenge 2: Solution Code -- Sorted Array Operations

```
#define MAX 100
int arr[MAX];
int arrSize = 0;
int arrComparisons = 0;
int arrShifts = 0;

// Insert into sorted array (maintain sorted order)
void arrInsert(int value) {
    // Find correct position
    int pos = 0;
    while (pos < arrSize && arr[pos] < value) {
        arrComparisons++;
        pos++;
    }
    if (pos < arrSize) arrComparisons++;  // final comparison that stopped the loop

    // Shift elements to the right
    for (int i = arrSize; i > pos; i--) {
        arr[i] = arr[i - 1];
        arrShifts++;
    }
    arr[pos] = value;
    arrSize++;
}
```

Challenge 2: Solution Code -- Array Search and Delete

```
// Binary search in sorted array
int arrSearch(int key) {
    int low = 0, high = arrSize - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        arrComparisons++;
        if (arr[mid] == key) return mid;
        if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;  // Not found
}

// Delete from sorted array
void arrDelete(int key) {
    int pos = -1;
    for (int i = 0; i < arrSize; i++) {
        arrComparisons++;
        if (arr[i] == key) { pos = i; break; }
    }
    if (pos == -1) { printf("  [Array] %d not found!\n", key); return; }

    // Shift elements left to fill the gap
    for (int i = pos; i < arrSize - 1; i++) {
        arr[i] = arr[i + 1];
        arrShifts++;
    }
    arrSize--;
}
```

Challenge 2: Solution Code -- BST with Counting

```
int bstComparisons = 0;

struct Node* bstInsert(struct Node* root, int value) {
    if (root == NULL) return createNode(value);
    bstComparisons++;
    if (value < root->data)
        root->left = bstInsert(root->left, value);
    else if (value > root->data)
        root->right = bstInsert(root->right, value);
    return root;
}

struct Node* bstSearch(struct Node* root, int key) {
    if (root == NULL) return NULL;
    bstComparisons++;
    if (key == root->data) return root;
    if (key < root->data) return bstSearch(root->left, key);
    return bstSearch(root->right, key);
}

struct Node* bstFindMin(struct Node* root) {
    while (root->left != NULL) { root = root->left; bstComparisons++; }
    return root;
}
```

Challenge 2: Solution Code -- BST Delete

```
struct Node* bstDelete(struct Node* root, int key) {
    if (root == NULL) return NULL;
    bstComparisons++;
    if (key < root->data) {
        root->left = bstDelete(root->left, key);
    } else if (key > root->data) {
        root->right = bstDelete(root->right, key);
    } else {
        // Found the node to delete
        if (root->left == NULL) {           // Case 1 & 2
            struct Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {   // Case 2
            struct Node* temp = root->left;
            free(root);
            return temp;
        }
        // Case 3: two children
        struct Node* succ = bstFindMin(root->right);
        root->data = succ->data;
        root->right = bstDelete(root->right, succ->data);
    }
    return root;
}
```

Challenge 2: Solution Code -- Main Program

```
int main() {
    printf("=== Challenge 2: BST vs Sorted Array Showdown ===\n\n");
    struct Node* root = NULL;
    int scores[] = {50, 30, 70, 20, 40, 60, 80, 45};
    int n = 8;

    // Phase 1: Insert all scores into both structures
    printf("--- Phase 1: Insert %d scores ---\n", n);
    for (int i = 0; i < n; i++) {
        arrInsert(scores[i]);
        root = bstInsert(root, scores[i]);
    }
    printf("[Array] Comparisons: %d, Shifts: %d\n", arrComparisons, arrShifts);
    printf("[BST]   Comparisons: %d\n\n", bstComparisons);

    // Phase 2: Search
    arrComparisons = 0; bstComparisons = 0;
    printf("--- Phase 2: Search for 40 and 55 ---\n");
    arrSearch(40); bstSearch(root, 40);
    arrSearch(55); bstSearch(root, 55);
    printf("[Array] Comparisons: %d\n", arrComparisons);
    printf("[BST]   Comparisons: %d\n\n", bstComparisons);

    // Phase 3: Delete
    arrComparisons = 0; arrShifts = 0; bstComparisons = 0;
    printf("--- Phase 3: Delete 30 ---\n");
    arrDelete(30);
    root = bstDelete(root, 30);
    printf("[Array] Comparisons: %d, Shifts: %d\n", arrComparisons, arrShifts);
    printf("[BST]   Comparisons: %d\n\n", bstComparisons);
    // ... (continued on next slide)
```

Challenge 2: Solution Code -- Main (continued)

```
    // Print final state of both structures
    printf("--- Final State ---\n");
    printf("Sorted Array: ");
    for (int i = 0; i < arrSize; i++) printf("%d ", arr[i]);
    printf("\n");
    printf("BST In-order: ");
    inorder(root);
    printf("\n");

    return 0;
}
```

Challenge 2: Expected Output

```
=== Challenge 2: BST vs Sorted Array Showdown ===

--- Phase 1: Insert 8 scores ---
[Array] Comparisons: 20, Shifts: 14
[BST]   Comparisons: 16

--- Phase 2: Search for 40 and 55 ---
[Array] Comparisons: 6
[BST]   Comparisons: 5

--- Phase 3: Delete 30 ---
[Array] Comparisons: 2, Shifts: 5
[BST]   Comparisons: 3

--- Final State ---
Sorted Array: 20 40 45 50 60 70 80
BST In-order: 20 40 45 50 60 70 80
```

Both produce the same sorted output -- but the BST did it with fewer total operations, and most importantly, **zero shifts**. The array had to shift elements 14 times just during insertion!

Challenge 2: Code Explanation

Part 1: Why Array Insertion is Expensive

When inserting **45** into the sorted array `[20, 30, 40, 50, 60, 70, 80]`:

```
Step 1: Find position (45 goes between 40 and 50)
        Compare with 20, 30, 40, 50 --> 4 comparisons

Step 2: Shift elements right to make room
        [20, 30, 40, _, 50, 60, 70, 80]
                        ^
        Shifted: 50, 60, 70, 80 --> 4 shifts

Step 3: Place 45
        [20, 30, 40, 45, 50, 60, 70, 80]
```

Every insertion requires shifting on average n/2 elements. For n insertions, that is O(n^2) shifts total.

BST insertion for 45:

```
50 -> go left -> 30 -> go right -> 40 -> go right -> NULL -> create leaf
3 comparisons, 0 shifts
```

No shifting needed because we just create a new leaf and attach it with a pointer.

Challenge 2: Code Explanation

Part 2: Deletion -- The Array's Hidden Cost

Deleting 30 from sorted array `[20, 30, 40, 45, 50, 60, 70, 80]`:

```
Step 1: Find 30 (linear scan) --> position 1, 2 comparisons

Step 2: Shift everything after position 1 to the left:
        arr[1] = arr[2] = 40     shift 1
        arr[2] = arr[3] = 45     shift 2
        arr[3] = arr[4] = 50     shift 3
        arr[4] = arr[5] = 60     shift 4
        arr[5] = arr[6] = 70     shift 5

        Result: [20, 40, 45, 50, 60, 70, 80]
        5 elements shifted!
```

BST deletion of 30 (a node with two children):

```
Find 30: 2 comparisons           (50 -> left -> 30)
Successor of 30 = 35             (smallest in right subtree)
Copy 35 into 30's position
Delete old 35 (a leaf -- trivial)
Total: 3 comparisons, 0 shifts
```

Challenge 2: Code Explanation

Part 3: The Final Scorecard

Operation	Sorted Array	BST
Insert 8 values	20 comparisons + **14 shifts**	16 comparisons + **0 shifts**
Search (2 queries)	6 comparisons	5 comparisons
Delete 1 value	2 comparisons + **5 shifts**	3 comparisons + **0 shifts**
Total work	**28 comparisons + 19 shifts**	**24 comparisons + 0 shifts**

The BST wins -- and the gap grows with more data. For n elements:

Operation	Sorted Array	BST (balanced)
Insert	O(n) due to shifting	**O(log n)**
Search	O(log n) binary search	**O(log n)**
Delete	O(n) due to shifting	**O(log n)**

Search is comparable, but insert and delete are where BSTs dominate. When data changes frequently (like a live leaderboard), BSTs are the clear winner.

Challenge 2: Code Explanation

Part 4: When Does the Array Win?

The BST is not always better. Here is the complete picture:

Scenario	Better Choice	Why
Data changes frequently (inserts/deletes)	BST	No shifting needed
Data is static (loaded once, searched many times)	Array	Binary search is faster (cache-friendly)
Need access by index (e.g., "5th largest")	Array	O(1) index access; BST would need O(n)
Data arrives in sorted order	Array	BST degenerates to O(n); array is already sorted
Memory is tight	Array	No pointer overhead (2 pointers per BST node)

The takeaway: Arrays with binary search are great when data is **static**. BSTs are great when data is **dynamic**. Choosing between them depends on how your data will be used.

Lab Session Complete!

Summary: What You Built

Challenge 1: Complete BST Implementation

- Built a BST from scratch with insert, search, findMin, findMax, and in-order traversal
- Inserted 10 scrambled values and got a perfectly sorted output -- no sorting algorithm needed
- Understood the recursive insert pattern: `root->left = insert(root->left, value)`

Challenge 2: BST vs. Sorted Array Showdown

- Implemented the same leaderboard operations using both a sorted array and a BST
- Counted comparisons and shifts to measure real work done
- Saw that arrays require O(n) shifts on insert/delete, while BSTs need O(log n) comparisons and zero shifts
- Learned when each approach is the right choice

Key Concepts Reinforced

The BST Property

- Left subtree < node < right subtree
- In-order traversal gives sorted output automatically

BST Operations Follow a Path

- Insert: walk down to find a NULL, create a leaf
- Search: walk down comparing at each node
- findMin/findMax: follow one side all the way down
- All are O(h) where h is the tree height

BST vs. Sorted Array

- Static data: sorted array with binary search wins
- Dynamic data: BST wins (no shifting)
- BST weakness: can degenerate to O(n) with sorted input (AVL/Red-Black fix this)

Complexity Summary

Operation	BST (balanced)	BST (worst)	Sorted Array
Insert	O(log n)	O(n)	O(n)
Search	O(log n)	O(n)	O(log n)
Delete	O(log n)	O(n)	O(n)
Find Min	O(log n)	O(n)	O(1)
Find Max	O(log n)	O(n)	O(1)
Print Sorted	O(n)	O(n)	O(n)
Access by Index	O(n)	O(n)	O(1)

Take-Home Challenge (Optional)

Extend your BST implementation with these exercises:

1. **Delete Operation:** Add the full BST delete function (all 3 cases: leaf, one child, two children). Delete values 20 and 50 from your Challenge 1 tree and verify the in-order traversal is still sorted.
2. **Height Calculator:** Write `int height(struct Node* root)` that returns the height of the BST. Insert the sorted sequence 1, 2, 3, 4, 5, 6, 7 and compute the height. Then insert the same values in the order 4, 2, 6, 1, 3, 5, 7 and compare heights. What do you observe?
3. **Think about it:** In Challenge 2, the BST performed well because the data was inserted in a shuffled order. What would happen if the scores arrived in sorted order (10, 20, 30, ...)? How many comparisons would each insertion take? How does this connect to the AVL trees we studied in lecture?